Trabajo Práctico 1 — *Circle of Life*

Contenidos requeridos

Para resolver este trabajo práctico, se requiere haber visto en clase los siguientes temas: - Tipos de datos simples. Variables. - Operadores aritméticos. Funciones built-in. f-strings. - Operadores lógicos y de comparación. Estructuras condicionales: `if`, `elif`, `else`. - Introducción a secuencias: cadenas, tuplas y listas. Indexing y slicing. - Intoducción a diccionarios. - Introducción a ciclos `while` y `for`. Sentencias `break` y `continue`. - Introducción a funciones. Scoping. Modularización.

Introducción

Usted es contratado por un parque nacional para estudiar la dinámica de un ecosistema compuesto por tres elementos: **pasto**, **conejos** y **zorros**. Se le pide que implemente un simulador que permita estudiar cómo evolucionan las poblaciones a lo largo del tiempo y bajo qué condiciones logran coexistir.

Imagen generada con Gemini

Modelo

El ecosistema se modela como una matriz de $N \times N$ celdas. Cada celda puede estar en uno de los siguientes estados:

- **Vacía**: no contiene ningún elemento.
- **Pasto**: una celda con vegetación que puede ser comida por conejos.

- **Conejo**: un herbívoro que se alimenta de pasto y es cazado por zorros.
- **Zorro**: un carnívoro que se alimenta de conejos.

Los conejos y los zorros se representan como **diccionarios** con las siguientes claves:

```
{
    "tipo":    "conejo" | "zorro",
    "energia": int,   # energía actual del animal
    "edad":    int    # cantidad de turnos que lleva vivo
}
```

La grilla se representa como una **lista de listas**, donde cada elemento es `None` (celda vacía) o uno de los diccionarios descritos arriba. Las celdas con pasto se representan con el string `"pasto"`.

Parámetros del modelo:

- N: tamaño de la grilla
- d_c: densidad inicial de conejos. Probabilidad de que una celda empiece con un conejo.
- d_z: densidad inicial de zorros. Probabilidad de que una celda empiece con un zorro.
- d_p: densidad inicial de pasto. Probabilidad de que una celda vacía (sin animal) empiece con pasto.
- e_c: energía inicial de los conejos y de cada conejo recién nacido.
- e_z: energía inicial de los zorros y de cada zorro recién nacido.
- g_c: energía que gana un conejo al comer pasto.
- g_z: energía que gana un zorro al comer un conejo.
- p_p: probabilidad de que en un turno dado el pasto crezca en una celda vacía adyacente.
- p_{rc}: probabilidad de que un conejo se reproduzca en un turno dado (si tiene energía suficiente).
- p_{rz}: probabilidad de que un zorro se reproduzca en un turno dado (si tiene energía suficiente).
- e_{min}: energía mínima necesaria para que un animal pueda reproducirse.

Vecindad

Se considera que dos celdas son **vecinas** si son adyacentes horizontal o verticalmente (vecindad de Von Neumann, 4 vecinos). Las celdas en los bordes de la grilla tienen menos vecinos.

Reglas de la simulación

En cada turno, el estado de la grilla se actualiza en cinco pasos. La propiedad central del modelo es que **todas las decisiones de movimiento se basan exclusivamente en el estado del turno anterior**, de modo que el resultado no depende del orden en que se procesan los animales.

Importante

Al comienzo de cada turno se debe guardar una **copia del estado actual** (snapshot). Todas las decisiones de movimiento deben consultarse sobre esa copia, no sobre la grilla que se está construyendo.

Paso 1 — Expansión del pasto

Para cada celda vacía (`None`) en el snapshot, si al menos una celda vecina tiene pasto, el pasto crece en esa celda con probabilidad p_p.

Paso 2 — Decisión de movimiento de cada animal

Cada animal elige su celda destino **consultando el snapshot**:

- **Conejo**: pierde 1 punto de energía. Si su energía llega a 0, muere (se registra su edad). Si sobrevive, elige aleatoriamente una celda vecina que **no tenga un animal en el snapshot**. Si no hay ninguna, se queda.
- **Zorro**: pierde 2 puntos de energía. Si su energía llega a 0, muere (se registra su edad). Si sobrevive, elige aleatoriamente una celda vecina que **no tenga un zorro en el snapshot**. Si no hay ninguna, se queda.

Paso 3 — Movimiento de los zorros

Se intentan mover los zorros en la nueva grilla a la direccion elegida por cada uno. Si un zorro no puede ocupar su destino (porque otro zorro llegó antes), se queda en su celda de origen.

Paso 4 — Movimiento de los conejos

Se intentan mover los zorros en la nueva grilla a la direccion elegida por cada uno. Para cada conejo:

- Si su celda destino **tiene un zorro** (en la nueva grilla): el zorro come al conejo, gana g_z puntos de energía, y se registra la edad del conejo.
- Si su celda destino está **libre**: el conejo se mueve. Si esa celda **tenía pasto en el snapshot**, el conejo lo come y gana g_c puntos de energía.
- Si su celda destino ya **tiene otro conejo**: el conejo se queda en su celda de origen.

Paso 5 — Reproducción

Después de moverse, si un animal tiene energía mayor o igual a e_{min}, se reproduce con probabilidad p_{rc} (conejos) o p_{rz} (zorros): aparece una cría con energía e_c o e_z en la **celda de origen** del progenitor, si esa celda quedó libre.

Fin de la simulación

La simulación termina cuando **no quedan conejos** o **no quedan zorros** en la grilla, o cuando se alcanza un número máximo de turnos T_{max}.

Números aleatorios

Para simular probabilidades, se puede generar un número aleatorio entre 0 y 1 con `random.random()` de la librería `random`. Si el número generado es menor o igual a la probabilidad dada, el evento ocurre.

Para elegir un elemento aleatorio de una lista, se puede usar `random.choice(lista)`.

Desarrollo

Se te pide que escribas los siguientes tres programas utilizando el lenguaje de programación Python, cada uno en un archivo separado. Se espera que las salidas de estos programas sean textos legibles, claros y bien formateados.

Los programas deben organizarse de la siguiente manera:

- `tp1_funciones_apellido.py`: contiene **todas las funciones** que implementan la lógica del simulador.
- `tp1_1_apellido.py`, `tp1_2_apellido.py`, `tp1_3_apellido.py`: importan las funciones del archivo anterior y resuelven cada consigna.

1 — Simulador del ecosistema

Se fijan los parámetros del modelo en:

- $N = 25$
- $d_c = 0.12, \quad d_z = 0.20, \quad d_p = 0.35$
- $e_c = 5, \quad e_z = 12$
- $g_c = 4, \quad g_z = 8$
- $p_p = 0.50, \quad p_{rc} = 0.18, \quad p_{rz} = 0.18$
- $e_{min} = 4$
- $T_{max} = 200$

Se te pide que escribas un programa que realice una simulación del ecosistema, **imprimiendo en la terminal** el estado de la grilla en cada turno. Junto con la grilla, se debe reportar el turno actual y la cantidad de conejos y zorros vivos.

Para imprimir la grilla, por cada celda se debe imprimir el string `"▓"` con el color correspondiente, o `"  "` (dos espacios) si la celda está vacía:

- `green` para el pasto
- `cyan` para los conejos
- `red` para los zorros

Recomendamos usar la librería `termcolor` para mostrar los colores en la terminal. Para instalarla:

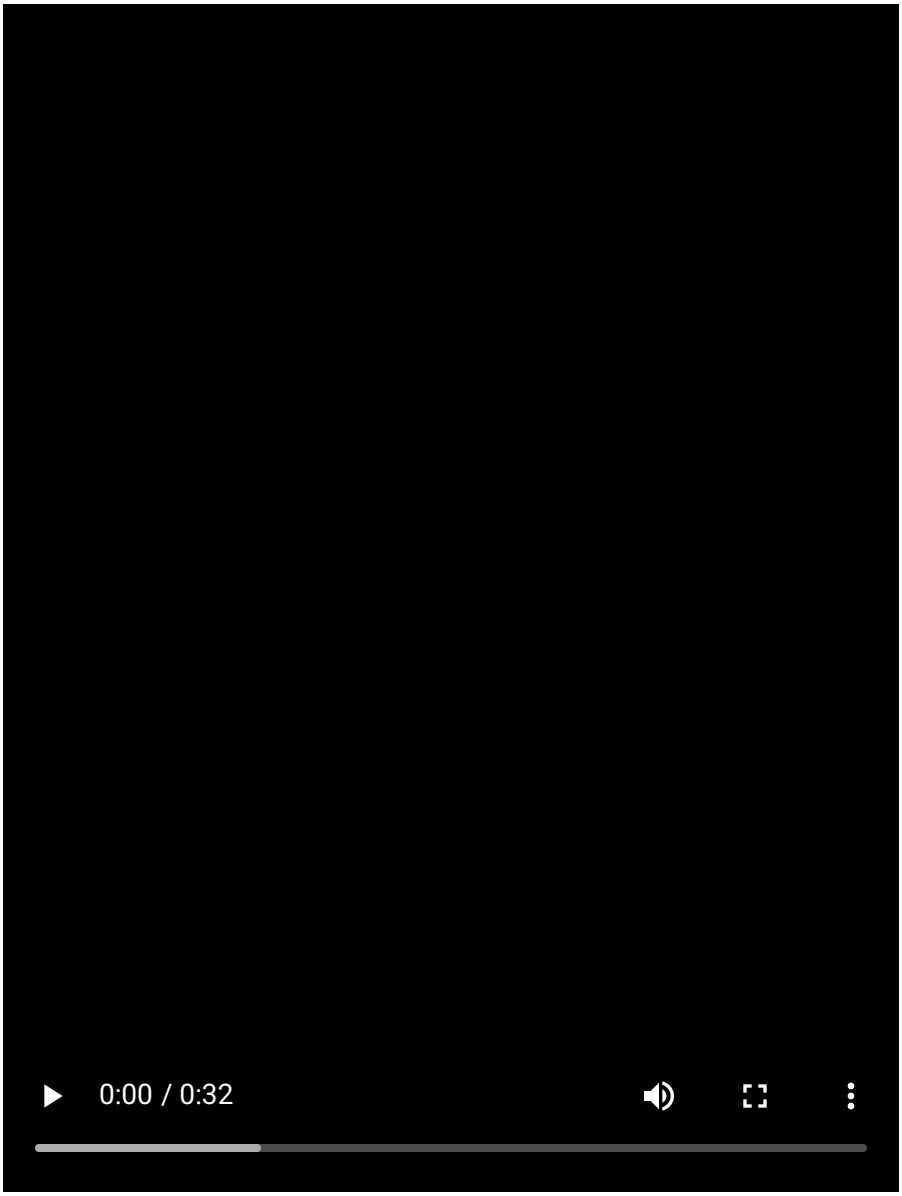
```
pip install termcolor
```

> ⚠ **Importante**
>
> Puede ser que deban usar `pip3 install termcolor` en lugar de `pip` para instalar la librería.

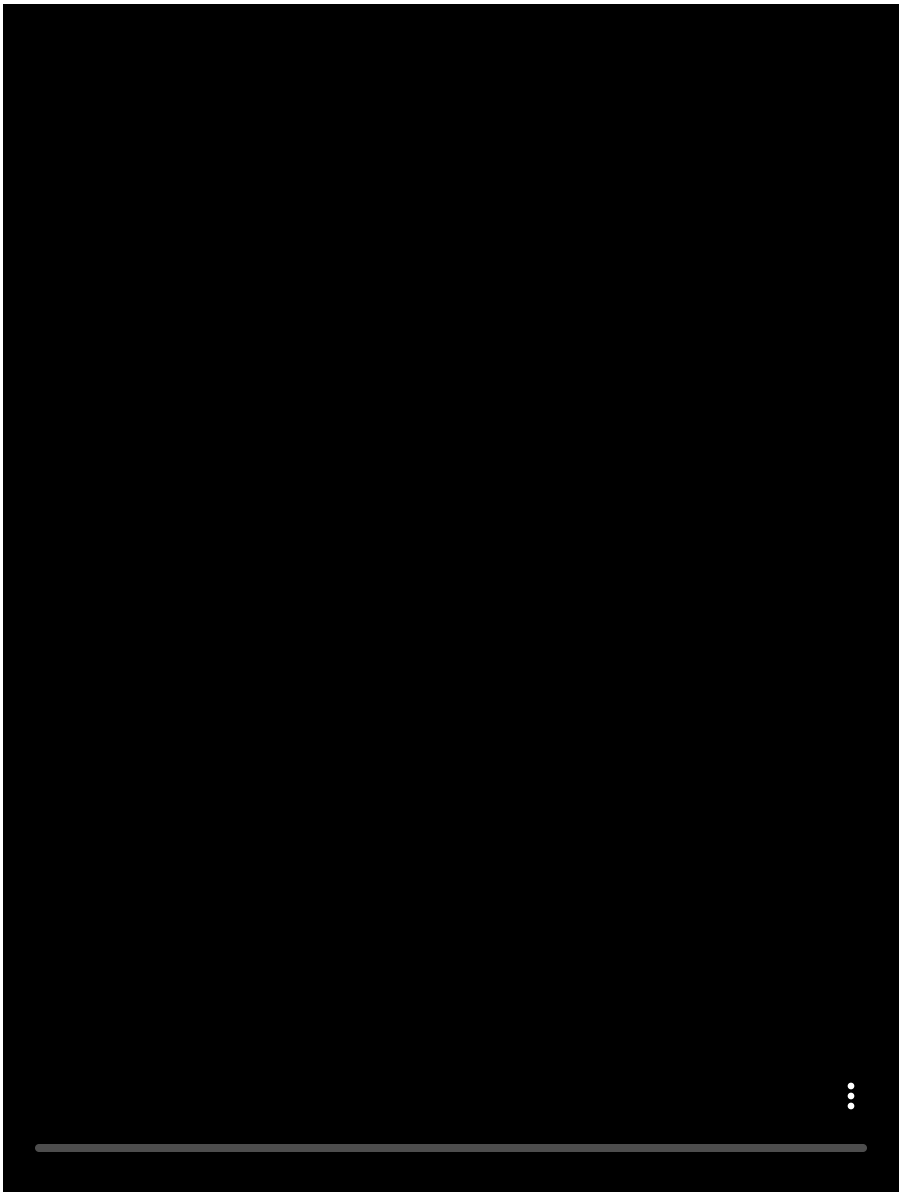
Recuerde dejar un tiempo de espera para que se pueda apreciar la evolución de la grilla. Se puede usar `time.sleep(segundos)` de la librería `time` para esto. Algo entre 0.1 y 0.3 segundos suele ser un buen valor.

Ejemplo de ejecución del programa

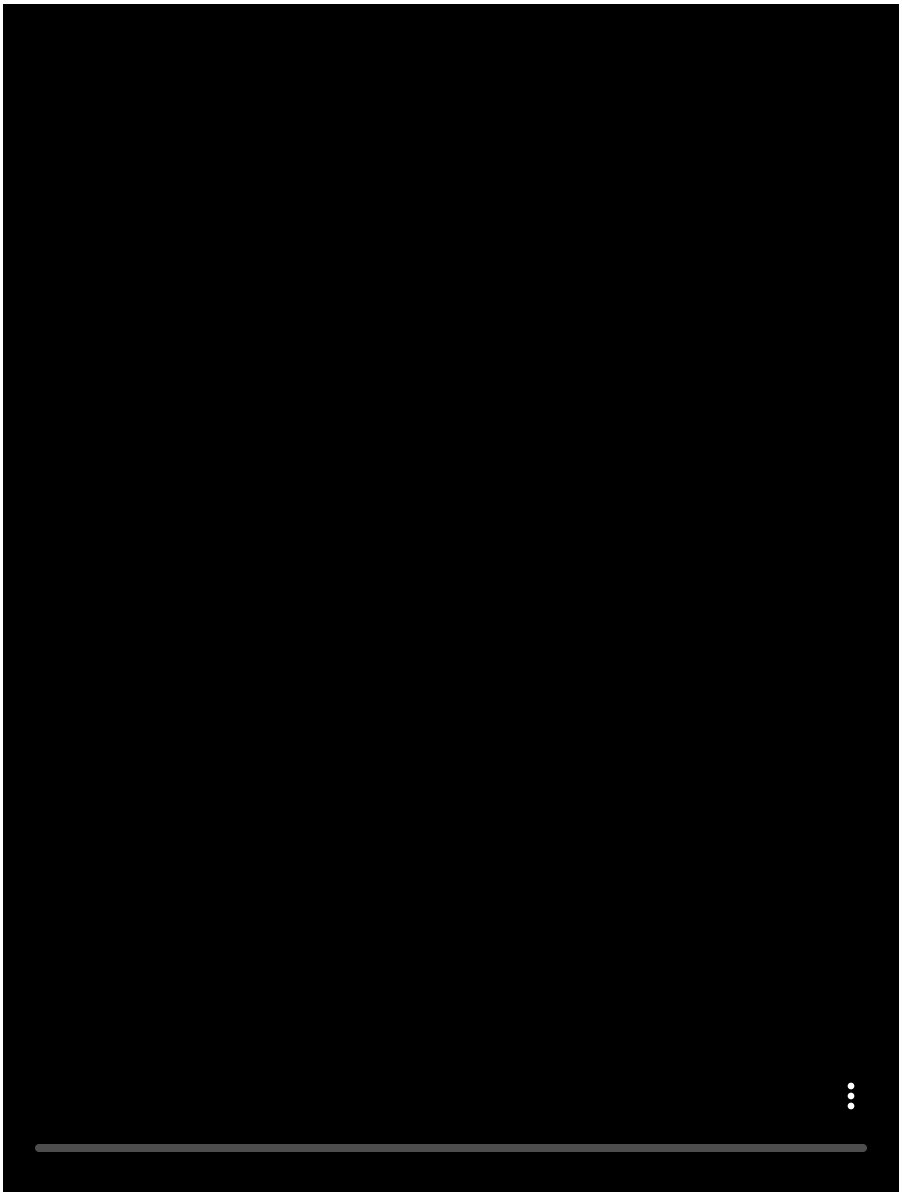
Simulación completa



Conejos extintos



Zorros extintos



Emojis

Para imprimir emojis en la terminal simplemente se puede hacer print de un string con el emoji. 🐇 🦊

2 — Esperanza de vida de los animales

Se utilizan los mismos parámetros que en el ejercicio anterior. La **esperanza de vida** de una especie se define como la **edad promedio a la que mueren** los individuos de esa especie a lo largo de la simulación. Recordá que la edad de un animal es la cantidad de turnos que lleva vivo, almacenada en la clave `"edad"` de su diccionario.

Se te pide que escribas un programa que ejecute **100 simulaciones** y que, acumulando las edades de todos los animales que murieron en cada simulación, calcule e imprima la esperanza de vida del conejo y del zorro.

El programa debe imprimir en la terminal únicamente tres líneas:

1. La cantidad de muertes registradas de cada especie (sobre todas las simulaciones).
2. La esperanza de vida del conejo.

3. La esperanza de vida del zorro.

No se debe imprimir nada más que el resultado en este programa.

¿Cuándo muere un animal?

Un animal muere cuando su energía llega a 0 al comienzo de su turno. En ese momento se debe registrar su edad antes de eliminarlo de la grilla. Los animales que siguen vivos al final de la simulación **no se cuentan** como muertes.

Salida ejemplo

```
Muertes registradas — Conejos: 139292 | Zorros: 99441
Esperanza de vida del conejo: 8.65 turnos
Esperanza de vida del zorro : 6.47 turnos
```

Importante

Los valores obtenidos en sus simulaciones pueden diferir de los del ejemplo, pero deberían ser parecidos.

3 — Impacto de la densidad inicial de zorros

Se sospecha que a mayor densidad inicial de depredadores, más inestable es el ecosistema y más probable es que alguna especie se extinga antes de T_{max}. Se te pide que escribas un programa que varíe d_z entre 0.01 y 0.39 (en pasos de 0.02) y que, para cada valor, ejecute **50 simulaciones** y calcule el **porcentaje de simulaciones en que no hubo ninguna extinción** (es decir, ambas especies coexistieron durante todos los T_{max} turnos).

Todos los demás parámetros se mantienen fijos en los valores del ejercicio 1.

El programa debe imprimir en la terminal únicamente una tabla con los resultados.

No se debe imprimir nada más que la tabla en este programa.

Salida ejemplo

```
+--------+---------------------+
| d_z    | Sin extinciones     |
+--------+---------------------+
| 0.01   |   78.00 %           |
+--------+---------------------+
| 0.03   |  100.00 %           |
+--------+---------------------+
| 0.05   |   98.00 %           |
+--------+---------------------+
| 0.07   |   96.00 %           |
+--------+---------------------+
| 0.09   |   98.00 %           |
+--------+---------------------+
| 0.11   |   92.00 %           |
+--------+---------------------+
| 0.13   |   98.00 %           |
+--------+---------------------+
| 0.15   |   90.00 %           |
+--------+---------------------+
| 0.17   |   84.00 %           |
+--------+---------------------+
| 0.19   |   68.00 %           |
+--------+---------------------+
| 0.21   |   64.00 %           |
+--------+---------------------+
| 0.23   |   40.00 %           |
+--------+---------------------+
| 0.25   |   18.00 %           |
+--------+---------------------+
| 0.27   |   26.00 %           |
+--------+---------------------+
| 0.29   |   14.00 %           |
+--------+---------------------+
| 0.31   |    8.00 %           |
+--------+---------------------+
| 0.33   |    6.00 %           |
+--------+---------------------+
| 0.35   |    8.00 %           |
+--------+---------------------+
| 0.37   |    4.00 %           |
+--------+---------------------+
| 0.39   |    2.00 %           |
+--------+---------------------+
```

Formato

La tabla debe estar formateada de la misma manera que en el ejemplo. Se recomienda usar f-strings para formatear los números.

Importante

Los porcentajes obtenidos en sus simulaciones pueden diferir levemente de los del ejemplo.

Entrega

El trabajo es individual. Se deben entregar los 4 archivos de Python a través de la tarea creada en el campus del curso. La fecha de entrega se encuentra en la misma tarea y en el calendario del curso.

Los nombres de los archivos deben ser:

- `tp1_1_apellido.py`
- `tp1_2_apellido.py`
- `tp1_3_apellido.py`
- `tp1_funciones_apellido.py`

El código debe estar implementado utilizando únicamente herramientas vistas en clase hasta el momento. No se permite usar librerías además de las mencionadas en el enunciado.

Se recuerda a los estudiantes que las entregas deben ser un producto original de cada estudiante, por lo que se les pide revisar la sección 6 del programa de la materia y el Código de Honor y Ética.

23 de marzo de 2026